

D213: Advanced Data Analytics

Task 2: Sentiment Analysis Using Neural Networks

Hannah R. Bergman

WGU Email: hbergm3@wgu.edu

Part I: Research Question.....	2
A1. Research Question.....	2
A2. Goals of the Data Analysis.....	3
A3. Neural Network Type.....	3
Part II: Data Preparation.....	3
B1. Exploratory Data Analysis.....	3
Presence of Unusual Characters (e.g., emojis, non-English characters).....	3
Vocabulary Size.....	3
Proposed Word Embedding Length.....	3
Statistical Justification for Maximum Sequence Length.....	4
B2. Tokenization Process.....	4
B3. Padding Process.....	5
Padding Process.....	5
Screenshot of a Single Padded Sequence.....	5
B4. Categories of Sentiment & Activation Function.....	6
Categories of Sentiment Used.....	6
Activation Function.....	6
B5. Data Preparation Steps.....	6
Step 1. Load & Inspect the Data.....	6
Step 2. Identify Unusual Characters.....	7
Step 3. Normalize the Text.....	8
Step 4. Tokenize the Reviews.....	8
Step 5. Determine Vocabulary Size.....	8
Step 6. Propose Word Embedding Length.....	8
Step 7. Review Word Count & Frequency.....	8
Step 9. Pad the Sequences.....	11
Step 10. Split the Data into Train, Validation, and Test Sets.....	11
B6. Prepared Dataset.....	12
Part III: Network Architecture.....	12
C1. Model Summary.....	12
C2. Layers, Types & Parameters.....	13
Total Number of Parameters.....	13
C3. Hyperparameters Justification.....	13

Activation Functions.....	13
Number of Nodes/Layer.....	14
Loss Function.....	14
Optimizer.....	14
Stopping Criteria.....	14
Evaluation Metric.....	14
Part IV: Model Evaluation.....	14
D1. Impact of Using Stopping Criteria.....	14
Number of Epochs.....	14
Screenshot of Final Training Epoch.....	15
D2. Model Fitness.....	15
Assessment of the Fitness of the Model.....	15
Actions Taken to Address Overfitting.....	15
D3. Visualizations.....	16
Model's Training Process.....	16
Line Graphs of Loss & Evaluation Metric (Accuracy).....	16
D4. Predictive Accuracy of the Trained Network.....	16
Part V: Summary & Recommendations.....	17
E. Code.....	17
F. Neural Network Functionality.....	17
Functionality of the Neural Network.....	17
Impact of the Network Architecture.....	17
G. Recommended Course of Action.....	18
Results.....	18
Recommended Course of Action Based on Results.....	18
Part VI: Reporting.....	18
H. Jupyter Notebook Report.....	18
I. Data & Code Sources.....	18
J. References.....	19

Part I: Research Question

A1. Research Question

Can we accurately predict the sentiment of a customer comment using previously labeled customer reviews from similar domains?

A2. Goals of the Data Analysis

The primary objective of this analysis is to determine whether the sentiment of a new customer comment can be accurately predicted using previously labeled reviews from similar domains. Using datasets from Amazon, Yelp, and IMDB that include both customer comments and sentiment ratings (positive [1] or negative [0]), our goal is to train a neural network model capable of classifying unseen comments based on patterns learned from past feedback.

To achieve this, the analysis will begin with cleaning and preprocessing the text data, followed by exploratory data analysis to understand sentiment distribution and commonly used terms within each class. The core of the project will involve training and evaluating a neural network-based classifier, using a relatively simple architecture, and measuring model performance based on accuracy.

A3. Neural Network Type

Model Type: Recurrent Neural Network with **LSTM (Long Short-Term Memory)**

LSTMs are a type of Recurrent Neural Network (RNN) well-suited for text classification and sequence modeling because they handle sequential data, are effective at capturing long-term dependencies (especially in longer reviews), and avoid the vanishing or exploding gradient problems common in vanilla RNNs (Starmer, 2022).

Part II: Data Preparation

B1. Exploratory Data Analysis

Presence of Unusual Characters (e.g., emojis, non-English characters)

Unusual (non-ASCII) characters in the dataset:
['\x85', '\x96', '\x97', 'â', 'é', 'ê']

Number of emojis found:
2,748

Vocabulary Size

The vocabulary size of the concatenated dataset is **5,402 unique words**, representing the number of distinct tokens after converting all text to lowercase and removing punctuation (without any further modification).

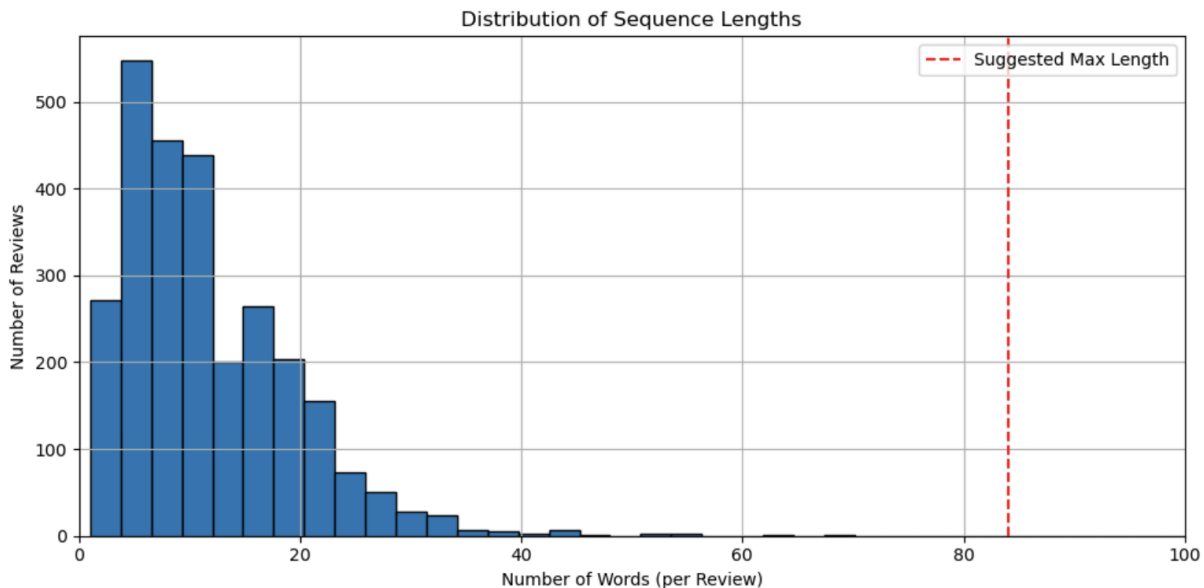
Proposed Word Embedding Length

My proposed word embedding length of **9 dimensions** adheres to the common guideline that embedding vectors should have a dimensionality equal to the fourth root of the vocabulary size (Sharma, 2024).

Statistical Justification for Maximum Sequence Length

Initially, I considered setting the maximum sequence length to 1,384, the length of the longest review in the dataset. However, to account for outliers, I also examined the mean (12.95) and standard deviation (35.68) of sequence lengths. Based on this, I've chosen a maximum sequence length of **84**. This was calculated by adding two to the mean sequence length and then multiplying that result by the standard deviation.

A plot of the sequence length distribution confirms this choice, showing that most reviews fall below the 84-word suggested max threshold.



B2. Tokenization Process

The goal of the tokenization process is to transform raw, free-form customer reviews into a structured, numerical format suitable for input into a neural network - in our case, specifically, an LSTM model. This begins with text normalization, where all text is converted to lowercase and stripped of punctuation or non-alphanumeric characters using Python's `re` module. This standardization reduces noise and ensures that variations in casing or symbols don't interfere with the model's learning.

```
# Normalize the text
def clean_text(text):
    text = text.lower() # uniformly convert text to lowercase
    text = re.sub(r"[^a-z0-9\s]", "", text) # strip punctuation, symbols & non-alphanumeric characters
    return text

combined_df["cleaned_review"] = combined_df['review'].astype(str).apply(clean_text)
ratings = combined_df['rating'].astype(int)
```

Next, TensorFlow's Tokenizer is used to map each word to a unique integer ID, turning each review into a sequence of tokens.

```
# Tokenize and convert to sequences
tokenizer = Tokenizer()
tokenizer.fit_on_texts(combined_df["cleaned_review"])
sequences = tokenizer.texts_to_sequences(combined_df["cleaned_review"])
```

B3. Padding Process

Padding Process

Once tokenized, text sequences will naturally vary in length. However, LSTM models require inputs to be of uniform shape. To solve this, we apply padding, which adds placeholder values (typically zeros) to each sequence until it reaches a predefined length.

We start by incrementing the `vocab_size` by 1 to account for the padding token. The tokenizer assigns an integer index to each unique word in the text, but when we use `pad_sequences`, it may introduce a 0 to represent padding (to make all sequences the same length). This padding value needs to be accounted for in the embedding layer, which expects the `input_dim` to cover all possible token indices (including 0), so adding +1 ensures the embedding matrix has room for the padding index.

```
# Pad vocab size
vocab_size = vocab_size + 1
```

To meet the uniform shape requirement, we have to standardize the number of tokens (words) in each review. Setting a reasonable `max_sequence_length`, based on our earlier suggested max sequence length value of 84, helps to prevent excessive padding while still retaining most of the content.

Finally, we apply padding to all sequences so they're exactly `max_sequence_length` long.

Padding is applied after the original text sequence (known as 'post' padding, `padding='post'`). This method preserves the beginning of each review while ensuring all sequences have the same length.

```
# Pad sequences to uniform length
max_sequence_length = suggested_max_seq_length # proposed max sequence length
padded_sequences = pad_sequences(sequences, maxlen=max_sequence_length, padding='post')
```

Screenshot of a Single Padded Sequence

The screenshot below shows a padded example of one tokenized review. The initial tokens correspond to the actual words in the review, followed by a series of 0s to fill the rest of the sequence up to 84 tokens. This ensures consistency in input shape while retaining the semantic order of words.

```

# Original review
print("Original review:\n", combined_df['review'][0])

# Cleaned review
print("Cleaned review:\n", combined_df['cleaned_review'][0])

# Tokenized review
print("Tokenized:\n", sequences[0])

# Padded Sequence Example
print("Padded Sequence:\n", padded_sequences[0])

Original review:
So there is no way for me to plug it in here in the US unless I go by a converter.
Cleaned review:
so there is no way for me to plug it in here in the us unless i go by a converter
Tokenized:
[27, 49, 5, 59, 118, 12, 73, 6, 367, 7, 11, 67, 11, 1, 183, 638, 3, 77, 62, 4, 2209]
Padded Sequence:
[ 27  49  5  59 118  12  73  6 367  7  11  67  11  1
 183 638  3  77  62  4 2209  0  0  0  0  0  0  0
 0  0  0  0  0  0  0  0  0  0  0  0  0  0
 0  0  0  0  0  0  0  0  0  0  0  0  0  0
 0  0  0  0  0  0  0  0  0  0  0  0  0  0]

```

B4. Categories of Sentiment & Activation Function

Categories of Sentiment Used

In this sentiment analysis project, the dataset is labeled for binary classification, distinguishing between **two categories: positive (label = 1) and negative (label = 0) sentiment**. These categories are derived from user reviews across platforms including Amazon, Yelp, and IMDB. Since there are only two sentiment classes, the model's output needs to reflect a binary decision - whether a review expresses a favorable or unfavorable opinion.

Activation Function

To support this, the final layer of the neural network uses a **sigmoid activation function**. The sigmoid function outputs a value between 0 and 1, which can be interpreted as the probability of the review expressing positive sentiment. If the output is greater than 0.5, the model predicts a positive sentiment; otherwise, it predicts negative sentiment. This setup also takes advantage of a binary cross-entropy loss function, which is specifically designed to optimize binary classification tasks.

B5. Data Preparation Steps

Step 1. Load & Inspect the Data

Combine multiple labeled datasets (Amazon, Yelp, IMDB) into a single dataset, saving it to CSV

```

# Load the data
amazon_df = pd.read_csv('amazon_cells_labelled.txt', sep='\t', header=None, names=["review", "rating"])
imdb_df = pd.read_csv('imdb_labelled.txt', sep='\t', header=None, names=["review", "rating"])
yelp_df = pd.read_csv('yelp_labelled.txt', sep='\t', header=None, names=["review", "rating"])

# Concatenate the datasets into one dataset
combined_df = pd.concat([amazon_df, imdb_df, yelp_df], ignore_index=True)

# Save combined dataset to CSV
combined_df.to_csv('d213_task2_combined.csv')

```

Preview the combined dataset

```
# Preview the concatenated (combined) data set
combined_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2748 entries, 0 to 2747
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0   review  2748 non-null     object
1   rating  2748 non-null     int64
dtypes: int64(1), object(1)
memory usage: 43.1+ KB
```

Ensure each entry includes a text review and a sentiment rating (0 = negative, 1 = positive)

```
# Ensure each entry includes a text review + sentiment (rating) label
combined_df.head()
```

	review	rating
0	So there is no way for me to plug it in here i...	0
1	Good case, Excellent value.	1
2	Great for the jawbone.	1
3	Tied to charger for conversations lasting more...	0
4	The mic is great.	1

Check for data imbalance by reviewing value counts

```
# Check for data imbalance
combined_df['rating'].value_counts()
```

```
1    1386
0    1362
Name: rating, dtype: int64
```

Step 2. Identify Unusual Characters

Identify the presence of unusual characters (e.g., emojis, non-English characters)

```
# Combine all text data into a single string for character-level analysis
all_text = " ".join(combined_df["review"].astype(str))

# Identify unusual (non-ASCII) characters – including potential emojis
unusual_chars = sorted(set(re.findall(r'[^\x00-\x7F]', all_text)))

# Function to detect if any emoji is present in a given text
def contains_emoji(text):
    return any(char in emoji.EMOJI_DATA for char in text)

# Apply emoji check across reviews
combined_df["has_emoji"] = combined_df["review"].astype(str).apply(contains_emoji)

# Count how many entries contain emojis
emoji_counts = combined_df["has_emoji"].value_counts()

# Display results
print("Unusual Characters:", unusual_chars)
print("\nEmoji Presence Count:\n", emoji_counts)
```

```
Unusual Characters: ['\x85', '\x96', '\x97', 'â', 'é', 'è']
```

```
Emoji Presence Count:
False    2748
Name: has_emoji, dtype: int64
```

Step 3. Normalize the Text

Convert all text to lowercase to ensure consistency. Remove punctuation, special characters, and non-alphanumeric symbols using regular expressions.

```
# Normalize the text
def clean_text(text):
    text = text.lower() # uniformly convert text to lowercase
    text = re.sub(r"^[a-z0-9\s]", "", text) # strip punctuation, symbols & non-alphanumeric characters
    return text

combined_df["cleaned_review"] = combined_df['review'].astype(str).apply(clean_text)
ratings = combined_df['rating'].astype(int)
```

Step 4. Tokenize the Reviews

Use Kera's Tokenizer to create a word-to-index dictionary. Convert each review into a sequence of integers based on the tokenizer.

```
# Tokenize and convert to sequences
tokenizer = Tokenizer()
tokenizer.fit_on_texts(combined_df["cleaned_review"])
sequences = tokenizer.texts_to_sequences(combined_df["cleaned_review"])
```

Step 5. Determine Vocabulary Size

Determine vocabulary size

```
# Vocab size
vocab_size = len(tokenizer.word_index)

print("Vocabulary Size:", vocab_size)
```

Vocabulary Size: 5402

Step 6. Propose Word Embedding Length

Propose a word embedding length

```
# Propose a word embedding length
embedding_dim = int(round(vocab_size ** 0.25)) # 4th root of vocab size

print("Proposed Word Embedding Length:", embedding_dim)
```

Proposed Word Embedding Length: 9

Step 7. Review Word Count & Frequency

Review word count of reviews to find the most frequently occurring words. Determine whether to retain or remove common “stopwords”.

```
# Word frequencies
top_words = pd.DataFrame(tokenizer.word_counts.items(), columns=["word", "count"]).sort_values(by="count", ascending=False)

# Top 30 (most frequent) words
top_words[:30]
```

Stopwords are words like “the”, “and”, “I”, “a”, “is”, etc. They often don’t contribute much to the meaning of a sentence when analyzing text (GeeksforGeeks, 2024), however, I ultimately decided to keep the stopwords in our dataset. Our specific task focuses on sentiment analysis, which relies heavily on retaining stopwords in order to comprehend the subtleties and emotional tone of text.

Additionally, our dataset is relatively small so there's no need to worry about improving processing time by removing stopwords to help reduce computational load (Singh, 2023).

	word	count
12	the	1940
71	and	1129
15	i	887
18	a	886
2	is	753
7	to	666
9	it	665
63	this	639
54	of	621
76	was	569
10	in	398
5	for	336
103	not	306
125	that	303
145	with	273
67	my	251
77	very	243
20	good	226
134	on	219
24	great	207
45	you	203
139	but	200
36	have	183
59	are	180
1907	movie	177
100	as	175
0	so	168
114	phone	162
1961	film	155
176	its	151

I also examined the last 10 words, checking to see if the last index number matched the vocab size value we determined earlier. Note that the last index value will be one digit less than the value of the vocab size since the index starts at 0.

```
# Bottom 10 words (previewing to double-check the last index matches vocab size)
bottom_words = top_words.tail(10)
print(bottom_words)
```

	word	count
3182	shooting	1
3184	completed	1
1444	linksys	1
3186	personalities	1
3188	twists	1
3189	occur	1
3190	irritating	1
3191	indescribably	1
3192	idiotic	1
5401	drawing	1

Step 8. Determine Maximum Sequence Length

Provide statistical justification for chosen maximum sequence length

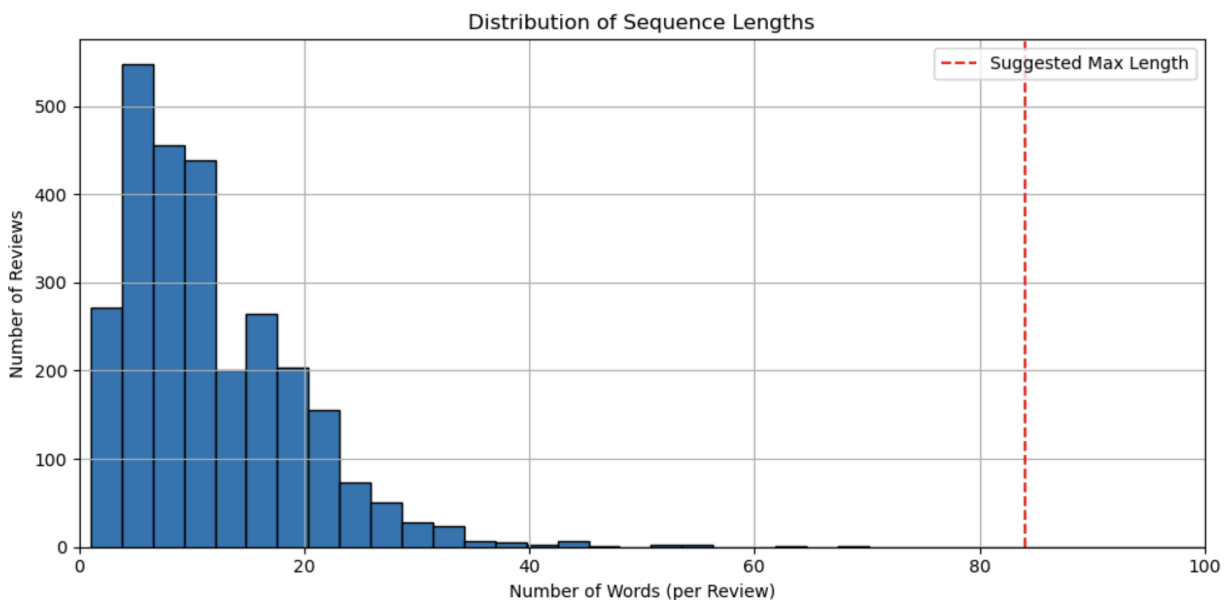
```
# Token counts
combined_df['token_count'] = [len(seq) for seq in sequences]

# Calculate summary statistics
max_seq_length = combined_df["token_count"].max() # max
mean_seq_length = combined_df["token_count"].mean() # mean
std_seq_length = combined_df["token_count"].std() # std
suggested_max_seq_length = round(mean_seq_length + 2 * std_seq_length) # suggested max using statistical rule of thumb

print(f"Max Sequence Length: {max_seq_length}")
print(f"Mean Sequence Length: {mean_seq_length:.2f}")
print(f"Std Dev Sequence Length: {std_seq_length:.2f}")
print(f"Suggested Maximum Sequence Length (mean + 2 * std): {suggested_max_seq_length}")
```

Max Sequence Length: 1384
Mean Sequence Length: 12.95
Std Dev Sequence Length: 35.68
Suggested Maximum Sequence Length (mean + 2 * std): 84

```
# Visualize distribution of sequence lengths
plt.figure(figsize=(10, 5))
plt.hist(combined_df['token_count'], bins=500, edgecolor='black')
plt.axvline(suggested_max_seq_length, color='red', linestyle='dashed', linewidth=1.5, label='Suggested Max Length')
plt.xlabel('Number of Words (per Review)')
plt.ylabel('Number of Reviews')
plt.title('Distribution of Sequence Lengths')
plt.legend(loc='upper right')
plt.xlim(0, 100)
plt.grid(True)
plt.tight_layout()
plt.show()
```



Step 9. Pad the Sequences

Use `pad_sequences()` to make all reviews the same length. Apply post-padding so zeros are added at the end of shorter sequences.

```
# Pad vocab size
vocab_size = vocab_size + 1

# Pad sequences to uniform length # proposed max sequence length
max_sequence_length = suggested_max_seq_length
padded_sequences = pad_sequences(sequences, maxlen=max_sequence_length, padding='post')
```

Print an example of a review from where it starts in its original state up to when it's transformed into a padded sequence.

```
# Original review
print("Original review:\n", combined_df['review'][0])

# Cleaned review
print("Cleaned review:\n", combined_df['cleaned_review'][0])

# Tokenized review
print("Tokenized:\n", sequences[0])

# Padded Sequence Example
print("Padded Sequence:\n", padded_sequences[0])
```

```
Original review:
So there is no way for me to plug it in here in the US unless I go by a converter.
Cleaned review:
so there is no way for me to plug it in here in the us unless i go by a converter
Tokenized:
[27, 49, 5, 59, 118, 12, 73, 6, 367, 7, 11, 67, 11, 1, 183, 638, 3, 77, 62, 4, 2209]
Padded Sequence:
[ 27  49   5  59 118  12  73   6 367   7  11  67  11   1
 183 638   3  77  62   4 2209   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0]
```

Step 10. Split the Data into Train, Validation, and Test Sets

Use `train_test_split()` from Scikit Learn to split the combined dataset: 70% training and 30% test. Further split the training set, again, opting for a 50/50 split for validation. The split sets are saved with a predetermined seed to ensure reproducibility for future experimentation.

```
# Prepare labels
X = padded_sequences
y = np.array(ratings)

# Split into training+validation and test sets (70/30)
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30, stratify=y, random_state=SEED, shuffle=True)

# Split training+validation into training and validation (50/50)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=SEED,
```

Convert the arrays to DataFrames, then save these train, validation and test sets to CSV

```
# Save to CSV
pd.DataFrame(X_train).assign(rating=y_train).to_csv("d213_task2_train_data.csv", index=False)
pd.DataFrame(X_val).assign(rating=y_val).to_csv("d213_task2_validation_data.csv", index=False)
pd.DataFrame(X_test).assign(rating=y_test).to_csv("d213_task2_test_data.csv", index=False)
```

B6. Prepared Dataset

The prepared datasets, 'd213_task2_combined.csv', 'd213_task2_train_data.csv', 'd213_task2_validation_data.csv', and 'd213_task2_test_data.csv' can be found within the main project directory and are also included as attachments to this report.

Part III: Network Architecture

C1. Model Summary

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 84, 100)	540,300
bidirectional (Bidirectional)	(None, 256)	234,496
dropout (Dropout)	(None, 256)	0
dense (Dense)	(None, 1)	257

Total params: 775,053 (2.96 MB)

Trainable params: 234,753 (917.00 KB)

Non-trainable params: 540,300 (2.06 MB)

This model is a sequential neural network built for binary sentiment classification, combining pretrained GloVe embeddings with a bidirectional LSTM architecture implemented in TensorFlow's high-level Keras API.

The model's initial layer is an embedding layer that transforms each word from a 5,403-word vocabulary into a 100-dimensional vector. I wanted to note that while an initial heuristic (fourth root of vocabulary size) suggested a 9-dimensional embedding, my testing demonstrated its major limitations for our baseline LSTM. This heuristic is more appropriate for randomly initialized embeddings trained from scratch on constrained datasets (Carruego, 2024). GloVe allows us to use pre-trained embeddings instead of learning embeddings from scratch so we're able to leverage word vectors that have already been trained on massive text corpora, capturing deep semantic relationships between words (Ahamed, 2018). Opting for the 100-dimensional GloVe vector gives the model richer and more informative word representations, improving performance without significantly increasing the risk of overfitting on our relatively small dataset - especially since the embeddings are frozen, meaning they're not updated during training.

Following the embedding layer, the model includes a bidirectional LSTM with 128 units in each direction, allowing it to capture contextual meaning from both past and future words in a sentence. This outputs a 256-dimensional representation of each review, which is passed through a dropout layer to help regularize the network by randomly disabling 50% of the neurons during training (Kashyap, 2024). Finally, the model uses a dense layer with a single sigmoid-activated output to return a probability between 0 and 1, indicating the predicted sentiment (positive or negative) of the review.

In total, the model has around 775,000 parameters, with about 234,000 trainable and the remaining 540,000 fixed in the frozen embedding layer. This design provides a balance between leveraging pretrained knowledge and retaining the flexibility to learn from the current task, making it effective and efficient for our use case - text classification with limited data.

C2. Layers, Types & Parameters

The model consists of **4 layers**:

Layer #1. Embedding Layer:

embedding (Embedding)	(None , 84, 100)	540,300
- Type: Embedding (GloVe 100 d, frozen) - This layer maps each token to a 100-dimensional vector using pre-trained GloVe embeddings. It's also frozen, meaning it doesn't update during training. - Parameters: 540,300, non-trainable		

Layer #2. Bidirectional LSTM Layer:

bidirectional (Bidirectional)	(None , 256)	234,496
- Type: Bidirectional LSTM (128 units x 2) - This layer processes the embedded input from both forward and backward directions to capture full sentence context. - Parameters: 234,496, trainable		

Layer #3. Dropout Layer:

dropout (Dropout)	(None , 256)	0
- Type: Dropout (0.5) - This layer randomly deactivates 50% of LSTM units during training to prevent overfitting. - Parameters: 0		

Layer #4. Dense Output Layer:

dense (Dense)	(None , 1)	257
- Type: Dense (sigmoid) - This layer outputs a single probability value between 0 and 1 for binary classification - Parameters: 257, trainable		

Total Number of Parameters

Total Parameters: 775,053

- **Trainable:** 234,753
- **Non-trainable:** 540,300 (from the frozen GloVe embedding layer)

C3. Hyperparameters Justification

Activation Functions

Our model's final dense layer uses a **sigmoid activation function** due to the nature of our binary sentiment classification task (positive vs. negative sentiment). The sigmoid function

outputs a probability between 0 and 1, making it ideal for predicting class membership in binary settings (Singhal, 2020).

Internally, the LSTM layers use both **tanh and sigmoid activation functions**. Tanh is specifically used for calculating the candidate cell states, which helps regulate the values flowing through the network. Sigmoid activations are important for the various gates within the LSTM (input, forget, and output gates). These gates control the flow of information, allowing the LSTM to selectively remember or forget information over long sequences. The combination of tanh and sigmoid within the LSTM's architecture helps effectively manage long-term dependencies and prevents issues like vanishing or exploding gradients during training (Vishal, 2024).

Number of Nodes/Layer

The **number of LSTM units was set to 128 per direction in a bidirectional structure**, enabling the model to capture both past and future word context for each review. The final dense layer has a **single output node**, reflecting the binary nature of the classification task.

Loss Function

The model uses **binary cross-entropy as the loss function**, which is a standard choice for binary classification problems. It measures the difference between the predicted probabilities and the actual binary class labels, effectively penalizing incorrect predictions (Godoy, 2018).

Optimizer

The model uses the **Adam optimizer**, which adaptively adjusts learning rates and tends to converge faster and more reliably than standard gradient descent (GeeksforGeeks, 2025).

Stopping Criteria

To prevent overfitting, **early stopping** was implemented with a **patience of 3 epochs**, halting training when validation loss stops improving.

Evaluation Metric

The model's performance is evaluated using **accuracy**, which is an appropriate metric here given the relatively balanced distribution of positive and negative sentiment labels.

Part IV: Model Evaluation

D1. Impact of Using Stopping Criteria

Number of Epochs

Using early stopping as a stopping criteria in training had a significant impact on both model performance and generalization. In our case, although we allowed training for up to 20 epochs, early stopping halted the training at epoch 13. This happened because the model's validation loss stopped improving for 3 consecutive epochs (as defined by patience=3). This early stopping mechanism ensures that training stops at the point where the model performs best on unseen data, rather than continuing to train and risk overfitting to the training set.

Screenshot of Final Training Epoch

Epoch 13/20
61/61 ————— 3s 44ms/step - accuracy: 0.9054 - loss: 0.2341 - val_accuracy: 0.8107 - val_loss: 0.4805

D2. Model Fitness

Assessment of the Fitness of the Model

The final bidirectional LSTM model demonstrates relatively strong overall fitness for our sentiment classification task. It achieved approximately 80% accuracy on the test set, with validation and test losses closely aligned, indicating consistent generalization to unseen data.

```
# Evaluate on test set
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print(f"\nTest Accuracy: {test_accuracy:.4f}")
print(f"Test Loss: {test_loss:.4f}")
print(f"Training stopped at epoch: {len(history.history['loss'])}")
```

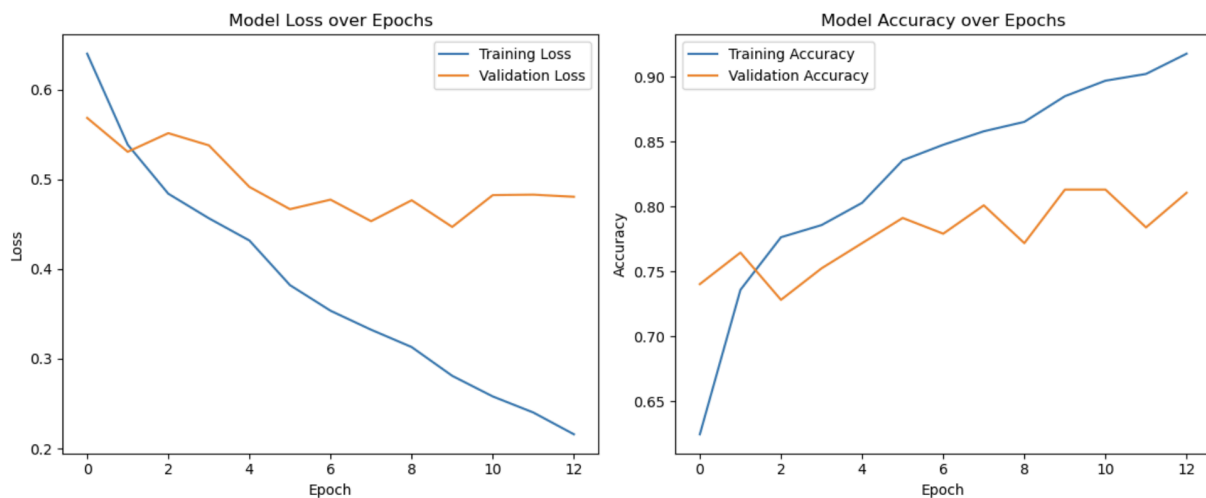
13/13 ————— 0s 19ms/step - accuracy: 0.8061 - loss: 0.4511

Test Accuracy: 0.8039

Test Loss: 0.4826

Training stopped at epoch: 13

The loss and accuracy curves show stable convergence, with both training and validation metrics improving together up until early stopping occurred at epoch 13. This balance between training and validation performance suggests that the model effectively learned meaningful sentiment patterns from the data without overfitting.



Actions Taken to Address Overfitting

Several steps were taken to prevent overfitting and improve the model's generalization. First, early stopping was implemented with a patience of 3 epochs, which stopped training as soon as validation loss stopped improving. A dropout layer (rate = 0.5) was also added after the LSTM layer to randomly deactivate half of the neurons during training, encouraging redundancy and discouraging reliance on specific features. Additionally, the model used pre-trained GloVe embeddings, which were frozen to prevent unnecessary parameter updates and reduce the

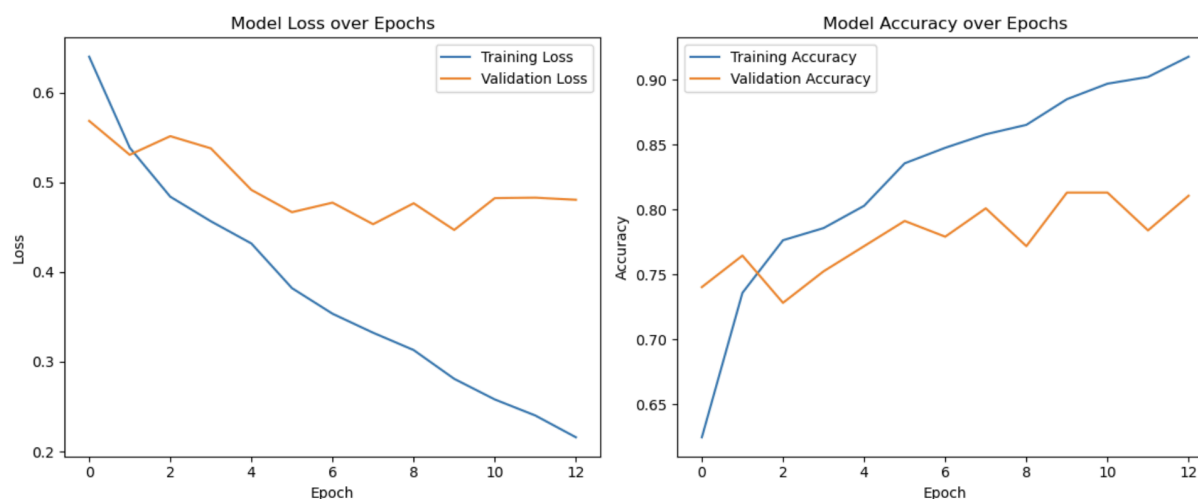
model's complexity. By limiting the maximum sequence length to a statistically reasonable cutoff (based on review length distribution), the model also avoided excess padding.

D3. Visualizations

Model's Training Process

```
Epoch 1/20  
61/61 ————— 10s 51ms/step - accuracy: 0.5650 - loss: 0.6737 - val_accuracy: 0.7403 - val_loss: 0.5684  
Epoch 2/20  
61/61 ————— 3s 44ms/step - accuracy: 0.7225 - loss: 0.5558 - val_accuracy: 0.7646 - val_loss: 0.5307  
Epoch 3/20  
61/61 ————— 3s 42ms/step - accuracy: 0.7776 - loss: 0.4939 - val_accuracy: 0.7282 - val_loss: 0.5514  
Epoch 4/20  
61/61 ————— 3s 43ms/step - accuracy: 0.7809 - loss: 0.4744 - val_accuracy: 0.7524 - val_loss: 0.5378  
Epoch 5/20  
61/61 ————— 3s 42ms/step - accuracy: 0.7969 - loss: 0.4493 - val_accuracy: 0.7718 - val_loss: 0.4915  
Epoch 6/20  
61/61 ————— 3s 43ms/step - accuracy: 0.8269 - loss: 0.4006 - val_accuracy: 0.7913 - val_loss: 0.4667  
Epoch 7/20  
61/61 ————— 3s 45ms/step - accuracy: 0.8423 - loss: 0.3666 - val_accuracy: 0.7791 - val_loss: 0.4773  
Epoch 8/20  
61/61 ————— 3s 45ms/step - accuracy: 0.8508 - loss: 0.3431 - val_accuracy: 0.8010 - val_loss: 0.4533  
Epoch 9/20  
61/61 ————— 3s 42ms/step - accuracy: 0.8633 - loss: 0.3167 - val_accuracy: 0.7718 - val_loss: 0.4766  
Epoch 10/20  
61/61 ————— 3s 45ms/step - accuracy: 0.8734 - loss: 0.2917 - val_accuracy: 0.8131 - val_loss: 0.4469  
Epoch 11/20  
61/61 ————— 3s 48ms/step - accuracy: 0.8893 - loss: 0.2744 - val_accuracy: 0.8131 - val_loss: 0.4824  
Epoch 12/20  
61/61 ————— 3s 44ms/step - accuracy: 0.8969 - loss: 0.2527 - val_accuracy: 0.7840 - val_loss: 0.4829  
Epoch 13/20  
61/61 ————— 3s 44ms/step - accuracy: 0.9054 - loss: 0.2341 - val_accuracy: 0.8107 - val_loss: 0.4805
```

Line Graphs of Loss & Evaluation Metric (Accuracy)



D4. Predictive Accuracy of the Trained Network

The predictive accuracy of the trained bidirectional LSTM model was evaluated using accuracy, which is a widely accepted metric for binary classification tasks - especially appropriate here, as our dataset is relatively balanced between positive and negative sentiment labels.

Using this metric, the model achieved an accuracy of approximately 80% on the test set, indicating that it correctly predicted sentiment (positive or negative) for 8 out of every 10 reviews. This level of performance demonstrates that the model was able to effectively learn and generalize sentiment-related patterns from the training data. The validation accuracy also closely mirrored training accuracy throughout the epochs, which suggests that the model's predictions are not only consistent during training but also stable on unseen data. In the context of sentiment analysis with relatively short, informal texts (reviews), an accuracy of 80% is a pretty strong result.

Part V: Summary & Recommendations

E. Code

Please refer to the attached file '**d213_task2.ipynb**' for the code used to save the trained network within the neural network.

F. Neural Network Functionality

Functionality of the Neural Network

The neural network is built to perform binary sentiment classification, predicting whether a review represents a positive or negative sentiment. We start by transforming raw text into numeric sequences that can be processed by the model. This is done through tokenization and embedding, where each word is converted into a vector representation. These vectors are then passed through layers that extract patterns and context from the sequence. The model produces a probability score between 0 and 1, which reflects the likelihood of the review being positive. Based on a decision threshold (0.5), the model then assigns the appropriate sentiment class.

This functionality enables the network to interpret text beyond simple word counts or keyword matching - it learns how combinations of words, word order, and context influence meaning. The model is capable of generalizing sentiment patterns from training data and applying them to new, unseen reviews with reasonable accuracy.

Impact of the Network Architecture

The architecture of the network directly influences its performance. The frozen GloVe embedding layer provides a dense, 100-dimensional vector for each word, capturing rich semantic information learned from large corpora. This eliminates the need to learn embeddings from scratch, which reduces training time and helps prevent overfitting. The bidirectional LSTM layer allows the model to read the sequence both forward and backward, capturing dependencies and context clues that might be otherwise missed by a unidirectional model.

To prevent overfitting, a dropout layer is applied after the LSTM to randomly deactivate a portion of the neurons during training. The dense layer with a sigmoid activation then condenses the sequence-level understanding into a single binary probability output. This architecture allows the model to handle natural language with a bit of flexibility and depth, balancing complexity with generalization.

G. Recommended Course of Action

Results

Our analysis answers our initial research question: Can we accurately predict the sentiment of a customer comment using previously labeled customer reviews from similar domains?

Through the development and evaluation of a bidirectional LSTM neural network, trained on a combined dataset of labeled reviews from Amazon, Yelp, and IMDb, we found that it is indeed possible to accurately predict sentiment. Our model achieved approximately 80% accuracy, with consistent performance across training and validation data. This suggests that sentiment patterns learned from one domain (movie reviews) can be generalized and applied effectively to others (product or service reviews), provided the language and tone share similar structures.

Recommended Course of Action Based on Results

The current model appears to generalize well across Amazon, Yelp, and IMDb data, based on the strong performance of the trained sentiment analysis model. We had ~80% accuracy across domains and while accuracy is a good baseline metric for a more balanced dataset, like ours, looking at accuracy alone isn't really enough. Accuracy can sometimes mask issues like false positives and false negatives, since accuracy doesn't tell us what kind of errors are being made, or performance across classes (it may hide how well or poorly the model performs on each class). Therefore, we should consider evaluating our model based on additional metrics, such as precision, recall, F1-score, and ROC-AUC, as well as build a confusion matrix to show actual vs. predicted sentiment to help visualize model mistakes and guide improvements going forward.

To further enhance model performance, we may also want to fine-tune the model on organization-specific customer reviews. This could involve collecting and labeling internal feedback (such as surveys, support tickets, or app store reviews) for specific domain adaptation. During this process, we might want to try unfreezing the embedding layer to allow the model to learn more domain-specific semantics. Additionally, extending the model's functionality to include a "neutral" class could help us capture more nuanced feedback.

Part VI: Reporting

H. Jupyter Notebook Report

Please refer to the attached file '**d213_task2.html**' for the HTML document of the executed Jupyter Notebook presentation.

I. Data & Code Sources

Kotzias, D. (2015). *Sentiment Labelled Sentences* [Dataset]. UCI Machine Learning Repository. Retrieved May 12, 2025, from <https://doi.org/10.24432/C57604>

J. References

- Ahamed, S. (January 13, 2018). *Text Classification Using CNN, LSTM, and Pre-Trained GloVe Word Embeddings: Part 3*. Medium.
Retrieved June 19, 2025 from medium.com/classifying-yelp-review-comments-using-cnn-lstm-and-pre-trained-glove
- Carruego, M. (February 15, 2024). *Choosing the Right Algorithm: Machine Learning vs. Heuristics*. Medium.
Retrieved June 19, 2025 from mikecarruego.medium.com/choosing-the-right-algorithm-machine-learning-vs-heuristics
- GeeksforGeeks. (January 3, 2024). *Removing Stop Words with NLTK in Python*.
Retrieved June 19, 2025 from geeksforgeeks.org/nlp/removing-stop-words-nltk-python
- GeeksforGeeks. (May 25, 2025). *What is Adam Optimizer?*
Retrieved June 19, 2025 from geeksforgeeks.org/deep-learning/adam-optimizer
- Godoy, D. (November 21, 2018). *Understanding Binary Cross-Entropy/Log Loss: A Visual Explanation*. Towards Data Science.
Retrieved June 19, 2025 from towardsdatascience.com/understanding-binary-cross-entropy-log-loss
- Kashyap, P (October 30, 2024). *Understanding Dropout in Deep Learning: A Guide to Reducing Overfitting*. Medium.
Retrieved June 19, 2025 from medium.com/@piyushkashyap045/understanding-dropout-in-deep-learning
- Kotzias et. al. (2015). *From Group to Individual Labels using Deep Features* [Academic Paper].
Retrieved May 12, 2025 from archive.ics.uci.edu/dataset/331/sentiment+labelled+sentences
- Sharma, S. (March 19, 2024). *Natural Language Processing Playlist – Chapter 3: Embeddings and word2vec*. Medium.
Retrieved June 16, 2025 from medium.com/@shantanu_sharma/natural-language-processing-nlp-playlist
- Singh, M.G. (August 29, 2023). *To Use or Lose: Stop Words in NLP*. Medium.
Retrieved June 19, 2025 from medium.com/@gelsonm/to-use-or-lose-stop-words-in-nlp
- Singhal, G. (September 9, 2020). *Introduction to LSTM Units in RNN*. Pluralsight.
Retrieved June 19, 2025 from pluralsight.com/resources/blog/guides/introduction-to-lstm-units-in-rnn
- Starmer, J. (July 11, 2022). *Recurrent Neural Networks (RNNs), Clearly Explained!!!* YouTube, uploaded by StatQuest.
Retrieved May 12, 2025 from youtu.be/AsNTP8Kwu80?si=P0As1P6rujUTumAn
- Vishal. (February 3, 2024). *Decoding LSTMs*. Medium.
Retrieved June 19, 2025 from medium.com/@vishal025/decoding-lstms